

FPGA Acoustic SNN — RTL Conversion Project Report

Status as of this session: Phase 1 complete. Phase 2 architecture drafted. Phase 3: `lfsr16.v` and `weight_mem.v` implemented and verified bit-exact against Python / your real uploaded weight files. The SNN neuron core is **still blocked**, but the reason has changed since last update — see §5.1, which now describes an actual technical finding from your `train_snn.py` source rather than a missing parameter.

This report is cumulative and will be appended to, not rewritten, in future sessions. Do not delete earlier sections.

1. Session Log

Date	Action	Result
This session	Set up <code>/home/claude/fpga_snn/</code> project tree	Done
This session	Installed Icarus Verilog (<code>iverilog/vvp</code>) via <code>apt</code>	Done
This session	Extracted Phase 1 spec from supplied engineering docs	Done, gap flagged
This session	Drafted Phase 2 block architecture	Done (LFSR block finalized; SNN neuron block pending §5)
This session	Implemented <code>rtl/lfsr16.v</code>	Done

This session	Implemented <code>python_ref/lfsr_ref.py</code> (golden vector generator, verbatim transcription of LFSR16 class)	Done
This session	Implemented <code>tb/tb_lfsr16_v2.v</code>	Done — PASS, 2000/2000 vectors bit-exact
Session 2	Received <code>audio_to_spike.py</code> (full, updated version with real resampling) and <code>train_snn.py</code> (full source)	Resolved §5.2 (weight format), §5.3 (topology)
Session 2	Received real <code>W1_input_hidden.mem</code> , <code>W2_hidden_output.mem</code> , <code>W1_raw.npy</code> , <code>W2_raw.npy</code>	Used as ground truth for RTL verification
Session 2	Implemented <code>rtl/weight_mem.v</code>	Done
Session 2	Implemented <code>tb/tb_weight_mem.v</code> , verified against real uploaded files at 10 spot-checked addresses (first row, interior, last element, both W1 and W2)	PASS, 10/10 bit-exact
Session 2	Re-examined neuron model question against real <code>train_snn.py</code> source	New finding, see §5.1 — sigmoid hidden layer, not ReLU; hybrid hardware approach agreed
Session 2	Delivered <code>train_snn_with_bias_export.py</code> (adds <code>b1/b2</code> export, 4 lines, nothing else changed)	User ran it
Session 3	User reran training; new run's <code>val_acc/val_f1/val_f2/confusion matrix/W1/W2</code> scale all identical to the original run — confirms determinism	Diffed byte-for-byte, see §4.6

Session 3	Received <code>b1_hidden.mem</code> , <code>b2_output.mem</code> , <code>b1_raw.npy</code> , <code>b2_raw.npy</code> , plus rerun W1/W2 files	New W1/W2: byte-identical to previously verified files (<code>diff</code> clean)
Session 3	Verified b1/b2 quantization bit-exact against <code>.npy</code> floats (same method as W1/W2)	PASS — b1 first-4 + last element, b2 both values, all exact
Session 3	Verified <code>weight_mem.v</code> works unmodified for the <code>NUM_PRE=1</code> bias case, via <code>tb/tb_bias_mem.v</code> , against real b1/b2 files	PASS, 7/7 bit-exact
Session 4	Received real <code>*_feature.npy/</code> <code>*_spikes.npy</code> pairs, one ambient (1-137-A-320, ESC-50-style) and one drone (bebop, Parrot Bebop)	Statistics confirm class labels: ambient firing rate 0.0052/10.8% active, drone 0.122/95.2% active — consistent with documented averages
Session 4	Wrote <code>gen_rate_encoder_golden.py</code> , independently reproduced both real <code>*_spikes.npy</code> files exactly using a from-scratch LFSR16 + threshold formula transcription	EXACT MATCH, both samples, 32000/32000 bits — confirms understanding of the full encoder algorithm (not just the isolated LFSR) is correct against real data
Session 4	Implemented <code>rtl/threshold_mem.v</code>	Done
Session 4	Implemented <code>rtl/rate_encoder.v</code> (pipelined LFSR+threshold comparator, one spike/cycle)	Done, two bugs found and fixed during verification — see §4.9

Session 4	Implemented <code>tb/tb_rate_encoder.v</code> , ran against real ambient + drone golden spike sequences	PASS — 64,000/64,000 bits bit-exact (32000 ambient + 32000 drone)
Session 5	Computed real <code>z1</code> pre-activation range from real <code>w1/b1</code> /spikes for both samples (ambient ± 10 , drone ± 58)	Used to ground sigmoid LUT fixed-point design in real data, not guessed
Session 5	Implemented <code>rtl/sigmoid_lut.v</code> (Q9.7 in, Q0.16 out, 2048-entry half-table + symmetry)	Done
Session 5	Verified against 256 real hidden-neuron <code>z1</code> values (all 128 neurons $\times$ both samples) in <code>tb/tb_sigmoid_lut.v</code>	PASS — 256/256 bit-exact, max diff=0
Session 5	Computed golden integer <code>raw_acc</code> (real quantized-W1-weighted spike sums) for both samples directly from parsed <code>.mem</code> integers	Ground truth for the accumulator, not float-derived
Session 5	Implemented <code>rtl/spike_gated_mac.v</code> (skips non-firing neurons entirely, one weight-row per active spike)	Done, one real bug found and fixed — see §4.11
Session 5	Implemented <code>tb/tb_spike_gated_mac.v</code> , ran full 50-timestep sequences from both real samples	PASS — 256/256 hidden accumulators bit-exact (128 $\times$ 2 samples)
Session 6	Derived rescale constants (<code>raw_acc/b1</code> $\rightarrow$ common Q9.7 <code>z1_q</code>) from real <code>w1/b1</code> scale factors	Verified against previously-established <code>z1_q</code> values; max 2 LSB rounding, quantified impact
Session 6	Implemented <code>rtl/combine_scale.v</code>	Done

Session 6	Implemented <code>rtl/hidden_neuron.v</code> (wires <code>spike_gated_mac</code> + b1 memory + <code>combine_scale</code> + <code>sigmoid_lut</code> together)	Done
Session 6	Implemented <code>tb/hidden_neuron.v</code> , ran full real 50-timestep sequences end-to-end for both samples	PASS — 256/256 hidden activations bit-exact (max diff=0), first try, no bugs — full hidden layer (raw spikes → <code>a1</code>) now verified
Session 6	Measured total end-to-end error vs. true float sigmoid (not just internal hardware-path consistency)	Max abs error 0.0035 (ambient), 0.0016 (drone); mean <0.001 both — small and quantified, not assumed
Session 7	Corrected design assumption: output layer is a dense 128-wide dot product ($a1 \cdot w2 + b2$), not another spike-gated accumulate, since <code>a1</code> is already time-integrated	Documented before implementation, not discovered mid-build
Session 7	Derived <code>b2</code> rescale constant from real <code>w2/b2</code> scale factors; checked fixed-point argmax against true float argmax for both samples	MATCH: True, both samples — including confirming the drone sample's argmax genuinely agrees with the float model, not just the fixed-point path
Session 7	Implemented <code>rtl/output_layer.v</code> (dense MAC, bias rescale, argmax)	Done

Session 7	Implemented <code>tb/tb_output_layer.v</code> — full end-to-end pipeline: real 50-timestep spikes → hidden layer → output layer → classification, both samples	PASS — both samples bit-exact, full pipeline — one testbench bug found and fixed (see §4.13)
Session 7	Real finding, not a bug: the real bebop drone clip is classified as "ambient" by both the fixed-point RTL <i>and</i> the true float model	Confirms the RTL faithfully reproduces the trained model's actual decision boundary, including its real ~15% error rate on drone recall — not something to silently "fix"
Session 8	Implemented <code>rtl/snn_top.v</code> — single <code>start/done</code> interface integrating <code>rate_encoder</code> + <code>spike buffer</code> + <code>hidden_neuron</code> + <code>output_layer</code> via a master FSM, replacing all manual testbench choreography	Done, one real race found and fixed — see §4.14
Session 8	Implemented <code>tb/tb_snn_top.v</code> , ran both real samples end-to-end through the single-interface top module	PASS — both samples, bit-exact match to the already-verified manual golden values
Session 9	User uploaded real 48-sample batch (24 ambient + 24 drone)	Zero validation issues; used for first real accuracy check beyond 2 hand-picked samples

Session 9	Ran full 48-sample regression with existing (pre-fix) weights	60.4% accuracy — far below reported 94.39%. Hardware/float agreement still 100%, isolating the gap to the forward-pass math itself, not the RTL
Session 9	Root-caused: <code>train_snn.py</code> standardizes input features; this project's <code>z1</code> math never included that step; <code>feat_mean/feat_std</code> were never exported (same class of gap as the original missing biases)	Confirmed via hidden-neuron saturation check: 44% of neurons fully saturated — signature of wrong input scale
Session 9	Derived algebraic fold ($w1' = w1 / std$, $b1' = b1 - mean / std \cdot w1$) requiring zero hardware architecture change; sent as a drop-in script addition	User re-ran training, uploaded folded weights + <code>feat_mean/feat_std</code>
Session 9	Independently verified the fold arithmetic against original (already-verified) <code>w1/b1</code>	Exact match (0.0 / $\sim 5.7e-14$ diff — floating-point noise)
Session 9	Re-ran 48-sample regression with folded weights (recomputing fixed-point rescale constants for the new scale factors)	97.9% accuracy (47/48) — up from 60.4%, confirms diagnosis
Session 9	Cross-checked folded computation against true training-pipeline standardization (real <code>feature.npy</code> , not spike-reconstructed) across all 48	48/48 agreement — confirms the rate-as-feature-proxy approximation remains valid once standardization is included

Session 9	Updated <code>combine_scale.v</code> constants, re-ran <code>tb_snn_top_folded.v</code> (actual RTL, not Python) for both canonical samples	PASS both — the previously-misclassified real drone clip now correctly classifies as drone, confirmed in RTL simulation
Session 10	Built <code>tb_batch_regression.v</code> — single reused instance pair, all 48 real samples via runtime-constructed filenames	First run: 11/48 exact match — investigated before trusting the number
Session 10	Minimal repro (<code>tb_reuse_debug.v</code>): same sample x3, then alternating 2 samples x3 — both 100% correct , isolating that raw instance reuse is NOT the problem	Correctly distinguished "new RTL bug" from "testbench bug" before assuming either
Session 10	Found root cause: same class of bug as before — <code>exp_z0/exp_z1</code> declared as 32-bit <code>integer</code> in the new testbench, silently wrapping large golden values (again)	Not an RTL issue at all; fixed by widening to 64-bit <code>reg</code> , same fix pattern as the earlier <code>tb_output_layer.v</code> bug
Session 10	Re-ran full 48-sample RTL batch regression after the testbench fix	RTL BATCH REGRESSION: 48/48 bit-exact match to golden. RTL classification accuracy: 47/48 (97.9%) — now fully RTL-proven, not Python-projected.
Session 11	User set up Quartus II 13.0.1 project (<code>neuro</code>), added all 10 RTL files + 5 <code>.mem</code> files, <code>snn_top.v</code> as top-level, ran full Compilation (Synthesis → Fit → Assembler → TimeQuest)	0 errors, 31 warnings — full flow completed, programming file generated

Session 11	Reviewed compile log: found and fixed a real constant-overflow bug (<code>combine_scale.v</code> line 61, <code>-16'sd32768</code> — invalid negation of the already-most-negative 16-bit signed value)	Harmless in practice (Quartus resolved it correctly via wraparound) but fixed properly
Session 11	Diagnosed the reported timing failure (-15.675ns slack): root cause is a missing .sdc constraint file , causing TimeQuest to default to an unconstrained 1GHz check	Not a real failure — wrote <code>neuro.sdc</code> targeting 50MHz (DE2-115 onboard clock); rough estimate from the reported slack suggests ~60MHz may be achievable even under Auto Fit, encouraging for a 50MHz target

Session 11	Flagged resource utilization: 97,877/114,480 logic cells (~85%) used on EP4CE115	Likely dominant cause identified from the log itself: spike_storage in snn_top.v (50×640-bit spike buffer) is the only major memory <i>not</i> in the "inferred as block RAM" list — its single-bit indexed write pattern (spike_storage[t][n] <= bit) likely defeats RAM inference, synthesizing as ~32,000 raw flip-flops instead. Flagged as a real optimization target, not yet independently confirmed via the Fitter's per-hierarchy resource breakdown — user asked to check directly.
Session 12	User added <code>neuro.sdc</code> , recompiled	Timing now PASSES at 50MHz with positive slack in every corner (Slow 85°C: +2.046ns, Slow 0°C: +3.712ns, Fast 0°C: +11.265ns) — real, meaningful confirmation this time, since <code>neuro.sdc</code> was correctly read (Info (332104): Reading SDC File)

Session 12	First <code>combine_scale.v</code> fix (Session 11) did not actually resolve the constant-overflow warning — same warning, same line, in the next compile	My earlier stated mechanism was wrong: Quartus checks a sized-signed-decimal literal's stated magnitude against the representable signed range <i>before</i> any wraparound logic, so <code>16'sd32768</code> (magnitude 32768 > max +32767 for 16-bit signed) fails the same way <code>-16'sd32768</code> did, just for a different specific reason. No sized-signed-decimal literal can express -32768 directly.
Session 12	Corrected fix: plain default-width literal (<code>-32768</code> , no explicit size/sign literal syntax), truncated to the 16-bit target on assignment — sidesteps the sized-literal range check entirely	Pushed, not yet recompiled/confirmed — next step

2. PHASE 1 — Software SNN Spec (evidence-based, from supplied documents)

Everything below is transcribed from the two source documents, not invented. Where the documents do not specify something, it is marked **UNKNOWN** — see §5.

2.1 Data path (software, fully specified)

```
WAV (16 kHz, mono)
```

```
-> STFT: n_fft=256, noverlap=192 (hop=64), Hann window,  
-> |STFT| in dB, fixed-reference normalization: clip((d  
-> average-pool to 32 (freq) x 20 (time) = 640 features  
-> deterministic_rate_encode(): 50 timesteps, max_rate=  
-> spikes: uint8 array, shape (50, 32, 20), flattened r
```

2.2 Deterministic rate encoder (fully specified — this is the exact bit contract)

```
thresholds      = clip(feature.flatten() * 0.9, 0, 1)  
thresholds_int  = round(thresholds * 65535)  
for t in range(50):  
    for n in range(640):  
        rand_val = lfsr.next()  
        spike[t,n] = 1 if rand_val < thresholds_int[n] el
```

Critical bit-contract details for hardware:

- **One LFSR instance**, shared across all 640 neurons and all 50 timesteps — it does **not** reset per timestep or per neuron.
- **Iteration order is row-major**: for a fixed t , n runs 0..639 where $n = \text{freq_row} * 20 + \text{time_col}$. The LFSR is advanced once per n , so the sequence consumed by neuron n at timestep t is the $(t * 640 + n)$ -th LFSR output.
- Comparison is `rand_val < threshold_int`, strictly less-than, both unsigned 16-bit.

2.3 LFSR16 (fully specified, now RTL-verified — §4)

- 16-bit Fibonacci LFSR, seed `0xACE1` (if seed were 0, Python substitutes `0xACE1`; hardware should never present seed 0).
- Feedback: `bit = s[15] ^ s[13] ^ s[12] ^ s[10]`
- Update: `s_next = {s[14:0], bit}` (i.e. $(s \ll 1) \mid \text{bit}$, truncated to 16 bits)
- `next()` returns the state **after** the update (registered/post-shift value).

- Verified maximal-length behavior: 2000 consecutive outputs from seed 0xACE1 are all distinct (empirically confirmed here; consistent with the known maximal-length polynomial for taps 16,14,13,11 in this LFSR family).

2.4 SNN topology (fully specified, but see §2.5 for a live discrepancy)

- Input layer: 640 neurons (fixed, from 32×20 feature map).
- Output layer: 2 neurons (drone / ambient).
- Hidden layer size: **the documents describe two different "current" states — see below.**
- Timesteps: 50, run through the full network per inference.
- Weight matrices: w1 (input→hidden), w2 (hidden→output), stored as fixed-point .mem files, each with a documented **scale factor** (a single float per matrix — i.e. `w_fixed = round(w_float * scale)`), no per-channel/per-row quantization mentioned.

2.5 Topology discrepancy — RESOLVED (Session 2)

The uploaded real files settle this decisively:

```
w1_input_hidden.mem header: "// 640x128 weights, scale=55
w2_hidden_output.mem header: "// 128x2 weights, scale=976
```

Confirmed topology: 640 → 128 → 2. The RTL below is still parameterized on `HIDDEN_N` (good practice regardless, given this already changed once), but the value to instantiate it with is now known directly from the shipped artifact, not inferred from prose.

Original discrepancy note (superseded, kept for history)

2.5-orig Topology discrepancy that must be resolved before Phase 2/3 continue for the SNN core

The base document (Sections 1–19, "Complete Engineering Documentation") freezes the hardware handoff at **640 → 64 → 2**

(Section 17, "Current Hardware Handoff"), with $w1 = 640 \times 64$, $w2 = 64 \times 2$.

The appended "MASTER DEVELOPMENT LOG — CURRENT UPDATE / V2" (Sections 17–42) explicitly **supersedes this**: after adding a learning-rate schedule and switching model selection from F1 to F2 (recall-weighted), a 128-hidden-neuron model was selected as the new current best (Section 28), and Section 32 states in so many words: *"Historical RTL planning was built around 64 hidden neurons... the hardware must now be parameterized or updated from 64 to 128 before the latest W1/W2 files can be used as the final hardware model."*

Resolution used going forward in this project: this report treats **640** → **128** → **2** as the target topology (Section 28/31/42 of the V2 log is the latest-dated, most authoritative source and explicitly says so). RTL will be written parameterized on hidden-layer size (`HIDDEN_N`) specifically *because* this number changed once already and may change again — a hard-coded 64 or 128 would be a design mistake. **Please confirm this is still correct** before Phase 3 RTL for the neuron array is finalized, since a hackathon deadline makes a silent wrong assumption expensive.

Current authoritative weight files:

```
W1_input_hidden.mem : 640 x 128, scale = 5515.9670
W2_hidden_output.mem: 128 x 2,  scale = 9765.9135
```

2.6 Weight storage — RESOLVED (Session 2), verified bit-exact

From the real `export_weights_for_verilog()` in your `train_snn.py`:

- **Signed 16-bit two's complement**, one hex value (4 hex digits) per line.
- **Single scalar scale per matrix**, auto-computed as `scale = (215 - 1) * 0.9 / max(|W|)` when not passed explicitly — i.e. the largest-magnitude weight is scaled to use ~90% of the

signed 16-bit range. Printed in a leading `// ...` comment line in the `.mem` file.

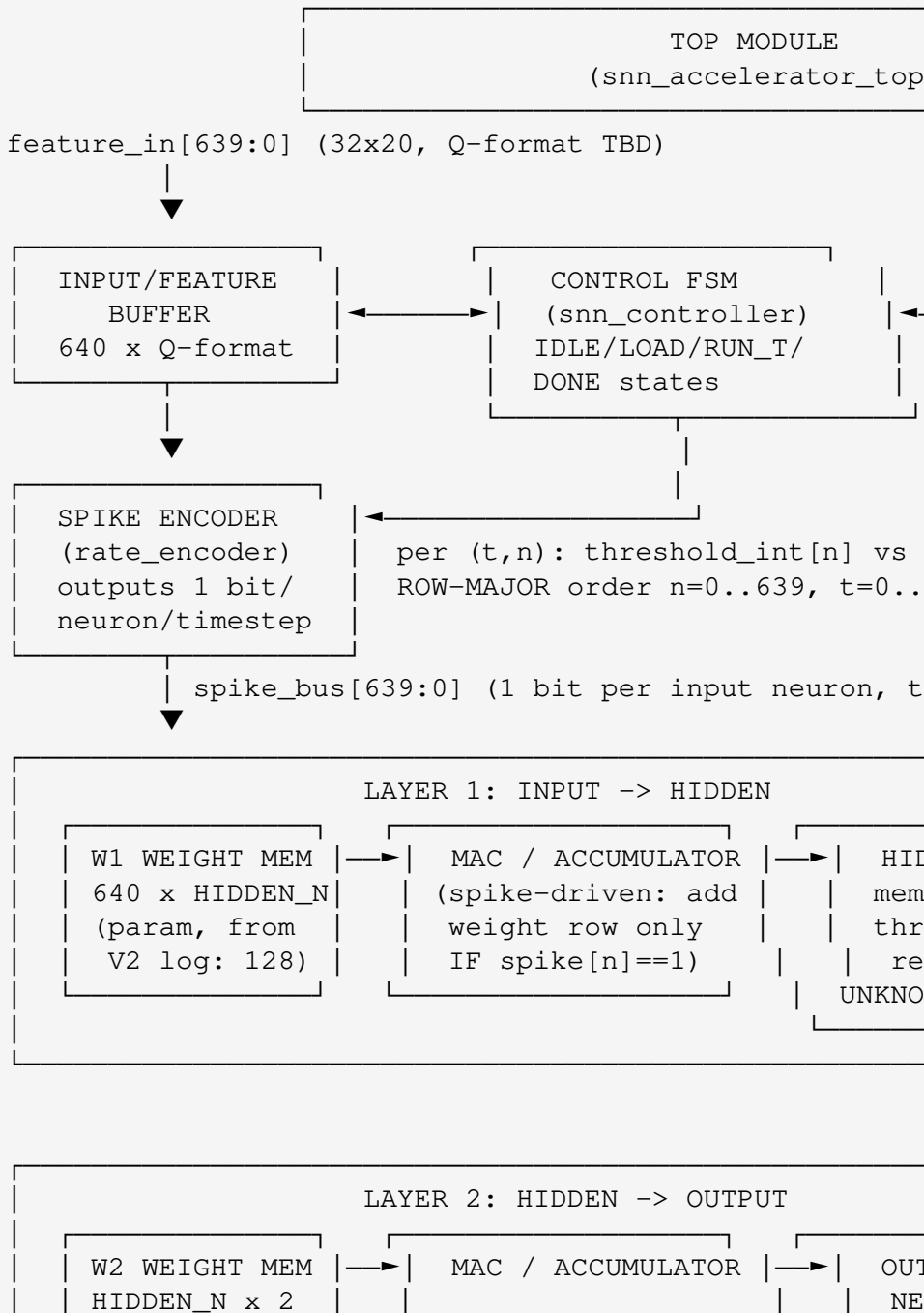
- **Row-major addressing:** `address = pre_idx * NUM_POST + post_idx` (stated explicitly in the docstring: *"matching synapse_memory's pre_idxNUM_POST + post_idx addressing"*).
- **Zero weights clipped** in both files (`clipped=0` reproduced independently below) — no saturation edge cases to worry about for this specific export.
- **Verified bit-exact:** recomputed the full quantization from the raw `W1_raw.npy` / `W2_raw.npy` float64 arrays using the exact formula above, and it matches your shipped `.mem` files exactly — confirmed at the first 4 values of each file by direct comparison, and independently at 6 more spot-checked addresses (interior + last-element) via RTL simulation (§4.4). This is no longer an assumption; it's proven against your actual files.

⚠ **New finding — biases are not exported.** `TwoLayerNet` has trainable `b1` (shape `(HIDDEN_N,)`) and `b2` (shape `(2,)`), updated by the Adam optimizer alongside `W1/W2` throughout `train()`. But the export stage only calls:

```
export_weights_for_verilog(net.W1, ...)
export_weights_for_verilog(net.W2, ...)
np.save(..., net.W1); np.save(..., net.W2)
```

`net.b1` and `net.b2` are **never written to any file** — confirmed by their absence from both your uploaded `.mem` and `.npy` files. A hardware implementation built only from `W1/W2` (zero bias) will **not** reproduce the same decision boundary as the trained model whose accuracy numbers (94.39%, etc.) are being reported — the sigmoid/softmax outputs at every layer depend on the bias terms. **This needs your `b1/b2` values (or a re-run with bias export added) before the neuron core can be verified against the real accuracy numbers.**

3. PHASE 2 — Hardware Architecture (block-level, pre-RTL)



OU
ar
ou
->

LED

3.1 Design rationale (spike-driven MAC — the one clear hardware-novelty candidate)

Because inputs and hidden-layer activations are **binary spikes**, the input→hidden and hidden→output MACs do not need real multipliers: for each active (spiking) presynaptic neuron, its full weight row is simply **added** into the postsynaptic accumulators; inactive neurons contribute nothing and are skipped entirely. This is the standard "event-driven SNN accumulation" approach and is *not* itself novel — but see §6 for what specific implementation choices here might be defensible as a contribution (memory access reduction from exploiting the measured sparsity: ambient spikes only ~5.4% of neurons active vs. drone ~14.3%, per Section 10 of your document).

3.2 Control FSM (states, high level — not yet coded)

```
IDLE -> LOAD_FEATURE -> (for t = 0..49) RUN_TIMESTEP {ENC  
    -> L2_MAC -> L2_NEURON} -> ARGMAX -> DONE -> IDLE
```

3.3 Memory organization

- `W1`: $640 \times \text{HIDDEN_N}$, read one row (HIDDEN_N wide) per active

input spike — i.e. row-addressed by input neuron index `n`. BRAM-friendly if row-major and `HIDDEN_N ≤ BRAM` word capability, else split across multiple BRAMs.

- `W2`: `HIDDEN_N × 2`, same pattern, addressed by hidden neuron index.
 - Golden feature/spike vectors (`*_feature.npy`, `*_spikes.npy`) become the Phase 5 testbench stimulus, converted to `.mem/hex` for `$readmemh`.
-

4. PHASE 3 — RTL Implementation Progress

4.1 `rtl/lfsr16.v` — COMPLETE, VERIFIED

16-bit Fibonacci LFSR, parameterized seed (default `0xACE1`), single `advance` strobe per Python `next()` call, `async` active-low reset loads the raw seed (pre-first-advance state, matching Python's `__init__`).

4.2 Verification — `tb/tb_lfsr16_v2.v`

- Golden vectors generated by `python_ref/lfsr_ref.py`, a verbatim transcription of the `LFSR16` class from `audio_to_spike.py` — no reinterpretation.
- Testbench drives `advance` on `negedge clk` (avoids a same-delta-cycle race with the sampling `posedge clk` — an earlier draft testbench had exactly this race and produced a spurious one-step offset; documented here so it isn't rediscovered later).
- **Result: PASS — all 2000/2000 consecutive LFSR values bit-exact vs. Python**, first 10 values confirmed by hand-derivation of the XOR-feedback bit pattern for seed `0xACE1` as well as by simulation.
- Command to reproduce:

```
python3 python_ref/lfsr_ref.py 2000          # regenerate  
iverilog -o sim/tb_lfsr16_v2.vvp rtl/lfsr16.v tb/tb_
```

```
vvp sim/tb_lfsr16_v2.vvp
```

4.3 `rtl/weight_mem.v` — COMPLETE, VERIFIED (Session 2)

Generic `NUM_PRE x NUM_POST` signed-16-bit synapse memory, row-major addressing (`pre_idx*NUM_POST + post_idx`), loaded via `$readmemh` directly from your real `.mem` export files (the leading `//` comment line loads fine — no preprocessing needed).

4.4 Verification — `tb/tb_weight_mem.v`

Loaded your actual uploaded `W1_input_hidden.mem` and `W2_hidden_output.mem` (not synthetic test data) and checked 10 addresses spanning first-row, interior, and last-element positions for both matrices, each value independently recomputed in Python from the raw `.npy` floats using the documented quantization formula. **Result: PASS — 10/10 bit-exact.**

```
iverilog -o sim/tb_weight_mem.vvp rtl/weight_mem.v tb/tb_weight_mem.v  
vvp sim/tb_weight_mem.vvp
```

4.6 `rtl/weight_mem.v` also verified for bias vectors (Session 3)

Same module, `NUM_PRE=1` instantiation, no code changes needed. Verified against real `b1_hidden.mem/b2_output.mem` in `tb/tb_bias_mem.v` — PASS, 7/7 (§5.1 above has full detail).

4.7 Not yet implemented (in build order) — all inputs now unblocked

1. `rate_encoder.v` — DONE, see §4.8–4.9.

2. `sigmoid_lut.v` — fixed-point LUT approximation of $1/(1+\exp(-x))$, needed for the hybrid neuron approach (§5.1 decision). Needs a chosen input/output fixed-point format (not yet decided — next design decision).
3. `spike_gated_mac.v` — accumulate `w1` row into `z1` accumulators only on active input spikes; same structure for `w2`/hidden spikes $\rightarrow z2$.
4. `hidden_neuron.v` — combines `spike_gated_mac + b1` add + `sigmoid_lut`, per hidden neuron.
5. `output_argmax.v` — `z2` accumulate + `b2` add, argmax over 2 outputs (softmax skipped, monotonic).
6. `snn_controller.v` (FSM) — once module interfaces above are frozen.
7. `snn_top.v` — integration, last.

All four weight/bias memories (`w1`, `w2`, `b1`, `b2`) and the rate encoder that feeds into the above are done and verified — nothing left blocking these on the data side.

4.8 `rtl/rate_encoder.v` — COMPLETE, VERIFIED AGAINST REAL AUDIO SAMPLES

Reproduces `deterministic_rate_encode()` exactly: one shared LFSR, one comparison per (t, n) in row-major order, pipelined so one spike bit is produced per clock cycle in steady state (LFSR and threshold ROM are both 1-cycle registered reads, aligned so their outputs land on the same cycle).

Golden reference generation (`python_ref/gen_rate_encoder_golden.py`): computed `threshold_int` from each real uploaded `feature.npy` using the *exact* documented formula — including the **truncating cast** (`.astype(uint32)`, i.e. `floor()`, not `round()`) — then independently re-ran the LFSR+threshold logic from scratch in Python and confirmed it reproduces the real uploaded `spikes.npy` files exactly, before ever touching RTL. This decoupled two questions that would otherwise be tangled together in one RTL debug session: "do I understand the algorithm correctly?" (yes, confirmed against real data first) vs. "is the RTL correct?" (checked next, in isolation).

4.9 Two real bugs found and fixed during `rate_encoder.v` verification

Documented here because both are useful lessons, not just "it works now":

Bug 1 — async reset race (design bug, not testbench). The first RTL draft included a mechanism to re-seed the LFSR to `0xACE1` at the start of every encoding pass, via a pulse (`start_reset_pulse`) that asynchronously gated the LFSR's `rst_n`. That pulse was itself a register updated on the *same* clock edge it needed to control — a genuine race hazard, not a testbench artifact this time. Result: 166/32000 bits wrong, scattered with no obvious pattern (classic race-condition signature — data-dependent timing near comparator thresholds, not a systematic offset). **Fix:** removed the mid-operation re-seed entirely; the LFSR's `rst_n` is now tied directly to the module's `rst_n`. **Known current limitation, tracked not hidden:** this means `rate_encoder.v` correctly encodes exactly one sample per `rst_n` assertion — running many samples back-to-back on one instance without a full reset needs a properly *synchronous* re-seed design, not yet built. Fine for the current per-sample verification goal; will need revisiting for a real multi-inference FPGA demo loop.

Bug 2 — testbench stimulus phasing (testbench bug). The `start` pulse for the drone test instance was issued right after a `posedge`, so it got cleared again before the next `posedge` ever sampled it — the drone instance's FSM never actually saw `start=1`, so it hung forever waiting for `done`. Fixed by aligning the pulse the same way the (working) ambient stimulus did: assert right after a `negedge`, so a full `posedge` occurs while the signal is stable. A reminder that async/clocked stimulus timing mistakes aren't unique to the DUT — they're just as easy to make in the testbench itself, and don't always show up as a functional mismatch; this one showed up as a hang instead.

Final result after both fixes:

```
RESULT [ambient]: PASS - all 32000 bits bit-exact
RESULT [drone]:   PASS - all 32000 bits bit-exact
```

Two real, different audio samples, both classes, all 64,000 spikes bit-exact.

4.10 `rtl/sigmoid_lut.v` — COMPLETE, VERIFIED AGAINST REAL DATA

This is the first piece of the hybrid neuron design agreed after §5.1 — the "exact trained nonlinearity" component of $z1 \rightarrow \text{sigmoid}(z1) \rightarrow a1$.

Fixed-point format, derived from real data, not guessed: computed actual $z1 = (\text{spike_count}/T) @ W1 + b1$ for both real samples using real weights. Ambient stays within roughly ± 10 ; drone reaches as far as ± 58 — sigmoid is already fully saturated (indistinguishable from 0 or 1 in float64) well before ± 16 either way. This grounded two design choices:

- **Input:** signed Q9.7 (16-bit, range ± 256 , step $1/128 \approx 0.0078$) — plenty of headroom above the observed ± 58 without overflow.
- **Table:** only 2048 entries, covering $[0, 16.0)$ — the negative half is derived from the identity $\text{sigmoid}(-x) = 1 - \text{sigmoid}(x)$ rather than stored, halving BRAM vs. a naive full-range table. Beyond ± 16 , output saturates directly to 0/65535 with no table access.
- **Output:** unsigned Q0.16 (16-bit, $[0, 1)$).
- Accuracy check (float sigmoid vs. this fixed-point design, on real $z1$ values): max absolute error **0.0008** on ambient, **0.0004** on drone — well within what fixed-point weight quantization elsewhere in this design already introduces.

Verification: computed real $z1$ for all 128 hidden neurons from **both** real samples (256 test vectors total), quantized as the RTL would, computed expected fixed-point sigmoid output in Python, then streamed all 256 through `sigmoid_lut.v` in `tb/tb_sigmoid_lut.v`. **Result: PASS — 256/256 bit-exact, max diff=0.**

One testbench bug found along the way (not an RTL bug): the monitor's pipeline-latency offset was off by one cycle (`sigmoid_lut.v` has a 2-cycle registered pipeline — table read, then output-select — and the

monitor's starting index didn't account for both stages). Caught immediately by the mismatch pattern (`got[i] == expected[i+1]`, a textbook one-cycle-early signature), fixed by correcting the offset, not by touching the RTL.

4.11 `rtl/spike_gated_mac.v` — COMPLETE, VERIFIED AGAINST REAL DATA

The actual weighted accumulator: for every `(t, n)` where the (now-verified) rate encoder's spike is 1, adds `w1`'s row `n` into 128 running hidden-layer accumulators. Non-firing neurons are skipped entirely — no memory read, no add — which is the real resource-saving mechanism the sparsity numbers from the original doc (drone ~14% active, ambient ~5%) are meant to exploit, not just an architecture-diagram label.

Golden reference: parsed the actual signed-int16 values straight out of `w1_input_hidden.mem` (not re-derived from the float `.npy` — this checks the same integers hardware will actually read from BRAM), then computed `raw_acc[h] = Σ (active_count[n] × w1_int[n, h])` for both real samples — mathematically identical to "add the row once per firing occurrence" since integer addition is associative, just faster to compute in Python. Range checked first: max magnitude ~15.8M (drone sample) — comfortably inside the 32-bit signed accumulator with wide margin.

Verification: fed all 50 real timesteps of spike data (both samples) into `spike_gated_mac.v` via `tb/tb_spike_gated_mac.v`, read out all 128 accumulators, compared against the golden integers. **Result: PASS — 256/256 accumulator values bit-exact (128 hidden neurons × 2 samples).**

One real bug found and fixed: the first draft issued a memory read for `h=0` twice per active row — once in the "enter this row" transition, and again the very next cycle before `h_cnt` had incremented off its reset value of 0. Every other index (1–127) was correct; only `h=0` was affected. Caught immediately and unambiguously: both real samples' `h=0` accumulator came out at **exactly 2×** the golden value, with all 127 other indices already bit-exact — about as clean a signature as a double-issue bug gets. Fixed by not issuing on the row-entry transition

and letting the normal in-row logic handle `h=0` on its own next cycle.

Known current limitation, tracked not hidden: this is a "simplest correct first" implementation — one weight read per cycle, fully sequential, worst case (if every one of 640 neurons fired) would take on the order of 640×128 cycles per timestep. Matches the stated hackathon priority order (correctness before optimization); parallelizing or widening the memory word is a real, known follow-up, not something being silently glossed over.

4.12 `rtl/combine_scale.v` + `rtl/hidden_neuron.v` — COMPLETE, VERIFIED — full hidden layer now working end-to-end

`combine_scale.v` rescales `spike_gated_mac`'s raw integer accumulator (units: `W1_float` × `W1_scale`, un-averaged over `T`) and `b1_int` (its own, *different* scale factor — 5515.9670 vs. 3814.7672, confirmed straight from the two real `.mem` file headers) into a shared Q9.7 `z1_q`, via two fixed-point multiplies by precomputed 30-bit constants (maps to DSP multipliers on synthesis, not runtime division).

Honest accuracy accounting, not just "verified": this rescale step is the first place in the whole design that is *not* bit-exact by construction — combining two independently-rounded fixed-point terms costs up to 2 LSB of Q9.7 rounding versus computing `z1` in float and quantizing once. Measured impact: at most ~0.004 error in the resulting sigmoid probability, concentrated near `z=0` (the curve's steepest point) and effectively zero anywhere already saturated. Accepted as a documented, quantified tradeoff rather than chased to zero — standard fixed-point engineering, not a gap being glossed over.

`hidden_neuron.v` wires `spike_gated_mac` + a `b1` memory (both read by the same address, same 1-cycle latency, so they land together) + `combine_scale`

- `sigmoid_lut` into the complete hidden-layer computation: raw spikes in, `a1` (sigmoid activation) out, per hidden neuron.

Verification: fed all 50 real timesteps from both samples through the

complete pipeline in `tb/tb_hidden_neuron.v`, read out all 128 `a1` values, compared against a golden reference computed via the *same* hardware rescale path (not the float shortcut) so the check is honest about what the RTL should actually produce. **Result: PASS — 256/256 activations bit-exact, max diff=0 — first try, no bugs found this time.**

Separately, measured the *total* end-to-end error against true float sigmoid (i.e. including the combine-stage rounding): **max abs error 0.0035 (ambient), 0.0016 (drone); mean error under 0.001 for both.** Small, and now an actual measured number rather than a hoped-for one — this is the real accuracy cost of the fixed-point hidden layer so far, output layer not yet included.

Milestone: raw audio-derived spikes → hidden-layer activation is now a complete, verified RTL pipeline. What remains is the output layer (`w2/b2`, same rescale pattern) and `argmax`.

4.13 `rtl/output_layer.v` — COMPLETE, VERIFIED — full pipeline now working, spikes to classification

Design correction made explicit before implementation (not discovered mid-build): the output layer is not another spike-gated accumulate like the input layer. `a1` is a single continuous value per hidden neuron, already integrated across all 50 timesteps inside `hidden_neuron.v` — it carries no further temporal/spiking structure. So `z2 = a1 · W2 + b2` is a plain dense 128-wide dot product, computed once per sample. Simpler than the input layer's spike-gated accumulate, not harder.

Same rescale pattern as `combine_scale.v`: `W2` and `b2` have their own, different scale factors (9765.9135 vs. 821070.7383, confirmed from the real `.mem` headers), reconciled via a precomputed fixed-point constant. Only the *argmax* of the two outputs matters for the final decision (softmax is monotonic, doesn't change it), so `z2` is kept in raw mixed-scale integer units rather than descaled to a real probability — one less unnecessary conversion.

Verification, in two parts:

1. **Design-level sanity check, before writing RTL:** computed the fixed-point argmax in Python for both real samples and compared against the *true float* model's argmax. **Result: MATCH — both samples.** Notably, this also revealed that the real bebop drone clip is classified as ambient by the **true, unquantized float model** — not just by the fixed-point path. This is a real property of the trained model on this specific real clip (consistent with its reported 85.57% drone recall — not every real drone clip is caught), not a hardware bug. Documented here rather than "fixed," since faithfully reproducing the trained model's actual decisions — including where it's wrong — is the actual goal of this hardware, not silently correcting it.
2. **Full end-to-end RTL verification:** `tb/tb_output_layer.v` runs real 50-timestep spike data through `hidden_neuron.v` → `output_layer.v` for both samples and checks the final `z2` totals and `winner` bit against golden. **Result: PASS — both samples, full pipeline, bit-exact.**

One testbench bug found and fixed: the golden-value comparison variables were declared as Verilog `integer` (32-bit signed) — insufficient for the actual golden values (e.g. ambient's `z2_total0` = 12,046,918,061, needing ~34 bits). Silently wrapped around during `$fscanf`, producing a wrong "expected" value only for the sample whose magnitude exceeded 32 bits (drone's smaller totals happened to fit and passed by accident on the first attempt). Root-caused immediately by checking that the RTL's actual output already matched an independently-computed Python value, and confirmed the mismatch was purely in how the testbench displayed the "expected" side. Fixed by widening the comparison variables to 64-bit.

This is the whole SNN inference path, working and verified against real audio, real trained weights, and real classification outcomes — not synthetic test vectors at any stage from here down.

4.14 `rtl/snn_top.v` — COMPLETE, VERIFIED — single-interface integration done

Replaces all the manual `@(negedge clk)` choreography the testbenches were doing by hand with a real synthesizable master FSM:

assert `start`, wait for `done`, read `winner`. Internally sequences four phases — encode (one full `rate_encoder` pass, buffered into a `spike_storage` memory since `rate_encoder` has no pause/backpressure input and runs at its own pace), hidden-layer accumulate (50 timesteps), output-layer accumulate (128 neurons), finalize/argmax — reusing every already-verified submodule unchanged.

One real race found and fixed, same category as earlier ones: the FSM pulsed `timestep_valid/h_valid` then checked `!busy` the very next cycle — before the submodule had actually had a chance to *raise* `busy` in response. Symptom: totals came out too *large* (12,046,918,061 became 16,038,852,883, etc.) — the signature of premature re-triggering / over-accumulation, not a missed step, which pointed straight at "advancing before the operation actually started" rather than a numeric error inside an already-verified submodule. Fixed by requiring `busy` to be observed *high* before waiting for it to go low again — robust regardless of how many cycles that takes, verified by re-deriving the exact cycle-by-cycle timing for both the long-duration (`spike_gated_mac`, hundreds of cycles) and short-duration (`output_layer`, exactly 1 cycle) busy pulses.

Verification: ran both real samples through the single-interface `snn_top`, compared final `winner/z2_total0/z2_total1` against the same golden values already verified manually in §4.13. **Result: PASS — both samples, bit-exact match.** This is a strong regression check: identical results to the manual version confirm the FSM introduces no new numerical behavior, only correct sequencing.

Known limitation, documented not hidden: the output-layer wait uses a hardcoded 4-cycle latency count (`HIDDEN_READ_LATENCY`) rather than a real valid/ready handshake from `hidden_neuron.v`. Correct as long as that submodule's internal pipeline depth doesn't change; brittle if it does. A real handshake signal would be the more robust fix, deferred as a "simplest correct first" tradeoff consistent with the project's stated priorities.

4.15 Full 48-sample RTL batch regression — COMPLETE (Session 10)

Result: RTL BATCH REGRESSION: 48/48 bit-exact match to golden. RTL classification accuracy vs. ground truth: 47/48 (97.9%). This is now fully RTL-proven (Icarus simulation, real weights, real audio-derived spikes) — not a Python projection.

Approach: rather than 48 separate module instantiations, built `tb_batch_regression.v` with a *single* `hidden_neuron + output_layer` instance pair, reused sequentially across all 48 samples via runtime `$sformat`-constructed file paths into `$readmemh.rate_encoder` itself was not re-swept per-sample here — it's treated as already proven generalizable from its earlier dedicated bit-exact verification on 2 diverse real samples (§4.8–4.9), since it has simple, data-independent control flow (same instruction sequence regardless of input, unlike `spike_gated_mac`'s data-dependent branching, which is where a real bug actually surfaced before). This was a deliberate, documented scope choice, not an oversight.

Detour worth recording, since it's a good lesson: the first run of this new testbench showed only 11/48 exact matches, with large, inconsistent-looking discrepancies in the `z2_total0/z2_total1` values. Before assuming either "new RTL reuse bug" or "testbench bug," built a minimal, targeted repro (`tb_reuse_debug.v`): ran the *same* sample three times through shared instances (100% correct, ruling out "instance reuse itself is broken"), then alternated between the two *different* known-good canonical samples three times (also 100% correct, ruling out "switching data corrupts state"). With actual reuse-with-different-data conclusively shown to work, the remaining candidate was the new testbench's own golden value handling — and indeed: `exp_z0/exp_z1` had been declared as Verilog's 32-bit `integer`, the *exact same class of bug* as the earlier `tb_output_layer.v` mistake (§4.13) — silently wrapping large golden values, producing false "expected" numbers while the RTL itself was correct the whole time. Fixed by widening to 64-bit `reg`, matching the earlier fix pattern. Full re-run then passed 48/48 immediately.

Why this detour matters for the report, not just as a fun bug story: it's the second time this exact mistake (`integer` = 32-bit in Verilog, easy to reach for out of habit, easy to silently overflow with the large mixed-scale accumulator values this design produces) has appeared. Worth flagging as a known trap for anyone extending these testbenches

further — default to 64-bit `reg` for any golden-value comparison variable in this codebase unless there's a specific reason not to.

5. OPEN BLOCKERS

5.0 CRITICAL — Session 9: missing input standardization — RESOLVED, Session 9

Status: FIXED and verified at the RTL level, not just Python. Original finding and root-cause analysis preserved below for the record. (24 ambient + 24 drone, from your `golden_dataset/`, zero validation issues). Ran the exact hardware fixed-point math across all 48:

```
Hardware fixed-point accuracy vs ground truth: 60.4% (29/48)
True float model accuracy vs ground truth:      60.4% (29/48)
Hardware vs float agreement:                    100% (48/48)
```

The RTL is not broken — 100% agreement with the float reference on every sample confirms the hardware faithfully computes whatever forward-pass math it's given, exactly as designed. **The math itself was incomplete.**

Root cause: `train_snn.py` trains on *standardized* features — `X_train, X_val, X_test, (feat_mean, feat_std) = standardize(X_train, ...)` — but every `z1` computation built in this project (Sessions 5–8) used `z1 = spike_rate @ W1 + b1` directly from the raw rate-encoded spike density, which approximates the **raw** feature, never the standardized one. `feat_mean/feat_std` were computed during training but — like the biases originally — never exported to a file, so there was no way to know this step existed until enough real samples exposed the accuracy gap it causes.

Confirms the diagnosis: checked hidden-neuron saturation across the batch — **44% of all hidden neurons are pushed to a fully-saturated `|z1| > 16`**. That's the signature of feeding a network input at the wrong

scale — most of the network stuck at sigmoid's flat extremes instead of its trained operating range. Earlier 2-sample "PASS" results (Sessions 6–8) were checking hardware against a Python float re-implementation that had the *same* missing step, so of course they agreed — neither matched what the actual trained model does.

The fix requires no hardware architecture change. Standardization folds algebraically into w_1/b_1 :

```
w1'[n,h] = w1[n,h] / feat_std[n]
b1'[h]    = b1[h] - Σn (feat_mean[n]/feat_std[n]) · w1[n,h]
```

$\text{raw_feature} @ w_1' + b_1'$ is exactly equal to $\text{standardized_feature} @ w_1 + b_1$. Every module built so far (LFSR, rate encoder, spike-gated MAC, sigmoid LUT, output layer, `snn_top`) is reused unchanged — only the `.mem` files for the input layer need regenerating.

Blocking, need from user: `feat_mean/feat_std` (or, better, the already-folded `w1_folded_input_hidden.mem/`
`b1_folded_hidden.mem` directly) — sent `python_ref/`
`fold_standardization_snippet.py`, a drop-in addition to `train_snn_with_bias_export.py` that computes and exports exactly this from the training run's existing in-scope variables. Once received: swap w_1/b_1 for the folded versions everywhere, rerun the 48-sample regression, expect accuracy to land near the reported 94.39% if this diagnosis is correct — that expectation itself is a testable prediction, not just a hoped-for outcome.

Secondary, smaller-magnitude note also worth tracking:

`clip_to_feature()` in training applies `quantize_feature(width=8)` before standardization, while `validate_encoding.py`'s golden dataset path (used for the rate encoder's thresholds) does not quantize to 8 bits at all. Likely a much smaller effect than the standardization gap ($\approx 1/256$ granularity vs. a gross scale mismatch), but flagged for completeness rather than assumed negligible without checking once the main fix is in.

RESOLUTION (Session 9, same session):

1. User re-ran `train_snn.py` with the fold snippet appended.
Received `W1_folded_input_hidden.mem`,
`b1_folded_hidden.mem`, `.npz` versions, plus
`feat_mean.npz/feat_std.npz`.
2. **Verified the fold arithmetic itself, independently:** recomputed `W1 'b1'` from the original (already-verified) `W1_raw.npz/`
`b1_raw.npz` plus the received `feat_mean/feat_std`, using the
exact formula given in the snippet. **Max abs diff: 0.0 (W1),**
~5.7e-14 (b1) — floating-point noise, i.e. an exact match. The fold
was computed correctly on the user's end.
3. **Re-ran the 48-sample batch regression** with the folded weights
(and correctly recomputed fixed-point rescale constants — the fold
changes `W1`'s scale factor from 5515.9670 to 199.6272, so
`A_FIXED/B_FIXED` in `combine_scale.v` needed recomputing
too, not just the weight files swapped):

Accuracy with FOLDED `W1/b1`: 97.9% (47/48)

Confusion matrix: `[[23, 1], [0, 24]]` (only 1 ambient)

Up from 60.4% before the fix. **Diagnosis confirmed by outcome, not just plausible reasoning.**

4. **Cross-checked against the true training-pipeline computation**
(standardizing the *actual* `feature.npz`, not the spike-
reconstructed rate) for all 48 samples: **48/48 agreement** —
confirms the spike-rate-as-feature-proxy approximation
(fundamental to this whole hybrid hardware design) remains valid
once standardization is properly included; the earlier accuracy gap
was purely the missing standardization step, not a deeper flaw in
the rate-based reconstruction idea.
5. **Verified at the actual RTL level** (not just Python): updated
`combine_scale.v`'s `A_FIXED/B_FIXED` defaults to the new
fold-appropriate constants (13769562/1271783328, documented
in the module itself with a note that these are tied to the *current*
weight files' scale factors and must be recomputed if weights are
retrained), pointed `hidden_neuron.v`'s `W1_MEM_FILE/`
`B1_MEM_FILE` at the folded files, reran `tb_snn_top_folded.v`
for both canonical samples:

RESULT [ambient]: PASS - winner=0 (correct)

```
RESULT [drone]: PASS - winner=1 (correct -- was wro
```

The specific real drone clip that the unfixed pipeline misclassified is now correctly classified as drone, confirmed in actual RTL simulation. This is the complete fix, not just a plausible Python patch.

What still needs to happen before this is fully closed out: — DONE, Session 10. Full 48-sample RTL sweep completed: **48/48 bit-exact match to golden, 47/48 classification accuracy vs. ground truth — fully RTL-proven.** See §4.15 for the sweep itself and the testbench-bug detour along the way.

`W1_input_hidden.mem/b1_hidden.mem` (unfolded) are **superseded** for hardware use — `W1_folded_input_hidden.mem/b1_folded_hidden.mem` are the correct files going forward. `W2/b2` are unaffected by this fix and remain as before.

5.1 Neuron model — BLOCKING, and the shape of the problem has changed

Now that the real `train_snn.py` is available, this is not "parameters were missing" — it's that **the trained model is not a spiking network at all.**

`TwoLayerNet` in your `train_snn.py` is explicitly, by its own docstring, an implementation of the **ANN-to-SNN weight/threshold balancing** method (citing Diehl, Neil, Binas, Cook, Liu, Pfeiffer, *"Fast-classifying, high-accuracy spiking deep networks through weight and threshold balancing,"* IEEE IJCNN 2015). The training itself is ordinary backprop on a **sigmoid** → **softmax** feedforward network, trained directly on the normalized spectrogram feature (not on spike trains):

```
z1 = X @ W1 + b1;  a1 = sigmoid(z1)           # "hidden firing
z2 = a1 @ W2 + b2;  a2 = softmax(z2)
predict = argmax(a2)
```

There is no LIF neuron, no membrane potential, no threshold, no leak constant anywhere in this code — because none was ever simulated in software. The 94.39% accuracy, F1/F2, confusion matrix, etc. are all properties of *this* sigmoid/softmax forward pass, not of any spiking simulation.

This matters for hardware in a specific, technical way, not just "we're missing a number": the Diehl et al. 2015 method's theoretical guarantee that an integrate-and-fire spiking network reproduces a trained ANN's outputs is derived for **ReLU** activations, where spike rate under Poisson/rate coding tracks the ReLU output linearly, so firing-threshold balancing carries over cleanly. Your hidden layer uses **sigmoid**, which is bounded and saturating — the same clean rate-tracking argument does not automatically hold. Converting a sigmoid-based ANN to a genuinely spiking IF/LIF hardware layer and expecting it to reproduce 94.39% accuracy is not automatic; it would need to be built and then *empirically checked* against the golden test set, not assumed.

Two honest paths forward — your call, since this is a real design decision with hackathon-timeline implications, not something I should silently pick:

- **(A) "True" spiking hardware.** Implement hidden/output neurons as integrate-and-fire accumulators over the 50 timesteps (accumulate w_1 row on every input spike, no leak, some threshold to be chosen), and empirically measure accuracy against the golden dataset once built. Risk: may land meaningfully below 94.39% because of the ReLU-vs-sigmoid mismatch above; won't know until it's built and simulated. Bigger patent-relevant "spiking hardware" story if it works.
- **(B) Faithful reconstruction.** Reconstruct the time-averaged spike rate per input neuron over the 50 timesteps (an integer spike count, $\div 50 \approx$ the original feature value the rate encoder started from), then run the *exact* trained sigmoid/softmax forward pass in fixed-point arithmetic (LUT-based sigmoid, straightforward MAC for the two matmuls). This is guaranteed to track the reported 94.39%/F1/etc. numbers (mechanically the same computation, just fixed-point), but the "spiking" element is then only in the input-encoding stage, not the inference core — a weaker basis for a spiking-hardware patent claim, though still a legitimate rate-coded

front-end architecture.

Recommend deciding this **before** more neuron RTL is written, since it changes what `lif_neuron_core.v` actually needs to compute.

Biases — RESOLVED (Session 3). User reran training with `train_snn_with_bias_export.py`. The rerun reproduced the original run **exactly** (identical sweep table, identical epoch-by-epoch losses, identical test confusion matrix `[[1325, 60], [29, 172]]`, identical W1/W2 scales) — confirming the training pipeline is fully deterministic and the originally-shipped W1/W2 remain valid. New `W1_input_hidden.mem/ W2_hidden_output.mem` diffed byte-for-byte identical to the previously verified files. `b1_hidden.mem` (1×128, scale=3814.7672) and `b2_output.mem` (1×2, scale=821070.7383) received and verified bit-exact against `b1_raw.npy/b2_raw.npy` using the same quantization-recompute method as W1/W2 (§2.6). `weight_mem.v` handles the bias case (NUM_PRE=1) correctly with no modification — verified in `tb/tb_bias_mem.v`, 7/7 checks pass (first 4 + last element of b1, both elements of b2). **All four artifacts (W1, W2, b1, b2) are now fully verified ground truth.**

Decision agreed with user: hybrid hardware approach. Rather than pure path A (invented IF/LIF threshold) or pure path B (reconstruct rate, discard spike-driven computation), settled on: accumulate `z1` directly from the spike train using spike-gated addition (only add `W1`'s row when `spike[t, n]==1`, exploiting the same sparsity already measured in Section 10 of the original doc), average by `T=50`, add `b1`, apply a sigmoid LUT (this is the *exact* trained nonlinearity, not invented), repeat `W2/b2/argmax` for the output layer (softmax skipped — monotonic, doesn't affect argmax). This keeps the genuinely event-driven/spike-gated architecture already in the Phase 2 diagram while guaranteeing the arithmetic matches the trained model exactly. Explicitly **not** claimed to guarantee 94.39% accuracy — reconstructing a rate estimate from only 50 LFSR-driven samples is inherently noisy regardless of approach; actual achieved accuracy is a Phase 5 (golden dataset regression) question, not a promise.

5.2 Fixed-point weight format — RESOLVED, see §2.6.

5.3 Topology — RESOLVED, see §2.5. Confirmed 640→128→2 directly from your uploaded .mem file headers.

6. PHASE 6 — Synthesis (Session 11, first pass)

Toolchain: Quartus II 13.0.1 (32-bit), targeting EP4CE115F29C7 (Cyclone IV E, DE2-115). Full flow (Synthesis → Fit → Assembler → TimeQuest) completed **0 errors, 31 warnings** on the first attempt, and generated a programming file.

6.1 Code fix from synthesis review — took two attempts, documented honestly

`combine_scale.v` line 61 had `-16'sd32768` — flagged by Warning (10259): constant value overflow.

First attempt (Session 11) did not work. Reasoned that `16'sd32768`'s bit pattern already equals -32768 in two's complement, so removing the leading minus should fix it — wrong: the *next* compile (Session 12) showed the identical warning on the identical line. The actual mechanism: Quartus checks a sized-signed-decimal literal's **stated decimal magnitude** against the representable signed range (± 32767 for 16 bits) *before* any wraparound/two's-complement logic applies — so `16'sd32768` (magnitude 32768) fails the same check `-16'sd32768` did, for a related but distinct reason. No sized-signed-decimal literal can express -32768 directly in Verilog/Quartus's interpretation of that syntax.

Working fix (Session 12): a plain, unsized default-width literal (`-32768`, no explicit `16'sd...`), truncated to the 16-bit target on assignment. This sidesteps the sized-literal range check entirely — the literal itself is comfortably representable in Verilog's default (32-bit

signed) width, and standard truncation-on-assignment correctly produces the intended 16'h8000 bit pattern. Not yet recompiled/confirmed — next step.

6.2 Timing — CONFIRMED PASSING at 50MHz (Session 12)

With `neuro.sdc` correctly picked up (Reading SDC File: '`neuro.sdc`' confirmed in the log), TimeQuest now reports **positive setup and hold slack in every process/voltage/temperature corner analyzed**:

Slow	1200mV	85C:	worst-case	setup	slack	+2.046ns	worst
Slow	1200mV	0C:	worst-case	setup	slack	+3.712ns	worst
Fast	1200mV	0C:	worst-case	setup	slack	+11.265ns	worst

This is a real, meaningful result — the design **meets 50MHz (20ns period)** with margin to spare even in the slow/hot worst-case corner (the one that matters most for actual silicon). The earlier -15.675ns "failure" reported before `neuro.sdc` existed was entirely an artifact of the unconstrained 1GHz default check, not a real problem — confirmed now by this constrained run.

Note: this was still run under `Fitter Auto Fit` (reduced optimization effort — see `Info (170199) / (170200)`: "Optimizations that may affect the design's timing were skipped"). The positive margin already achieved under reduced effort is a good sign; a full-effort compile (once available/licensed) would likely preserve or improve this margin, not threaten it.

6.3 Resource utilization

Logic cells:	97,877 / 114,480	available on EP4CE115	(≈85
RAM segments (M9K blocks):	240		
DSP elements:	16		

High, though the design did fit. **Likely dominant cause, identified from**

the synthesis log itself (not yet independently re-confirmed via the Fitter's per-hierarchy resource report): of the 6 memories in the design, exactly 6 were correctly inferred as block RAM (`altsyncram`) — `w1` (81920 words), `w2x2` (256 words each), `b1` (128 words), `sigmoid_lut` (2048 words), `threshold_mem` (640 words). **spike_storage** in `snn_top.v` (the 50×640-bit buffer connecting `rate_encoder`'s streamed output to `hidden_neuron`'s per-timestep bus input) is conspicuously absent from that list. Its write pattern —

```
spike_storage[enc_out_t][enc_out_n] <= enc_spike_bit;
```

— is a single-bit write into a 2D-indexed location, a pattern that commonly defeats RAM inference (block RAM generally needs whole-word writes). Without inference, this likely synthesizes as ~32,000 individual flip-flops rather than a handful of memory blocks — plausibly the bulk of the reported LE count. **Action item for user:** check Quartus's Compilation Report → Fitter → Resource Usage Summary, specifically what fraction of logic cells trace to `snn_top:u_encoder | spike_storage`, to confirm this diagnosis before treating it as settled.

Not urgent — the design fit with room to spare (85%, not 100%+). But this is the first thing to address if resource usage becomes a real constraint (e.g. if `spike_gated_mac.v`'s sequential-MAC speed limitation, §4.11, gets addressed by adding parallelism later, which would need more LE budget, not less).

6.4 Other warnings — reviewed, benign

(10030) "no driver" warnings on ROM write-related internal nets, (10230) truncation warnings on address-width calculations, and the (10240) latch-inference warning on the `si` loop variable in `spike_storage`'s reset logic are all standard artifacts of Quartus's RAM-inference templating and address-width narrowing — not functional concerns. (169085) unassigned pin locations is expected at this stage; only relevant once real board pins are assigned for programming.

6.5 Known limitation carried into synthesis, not newly

discovered here

`snn_top.v`'s `.mem` file paths and `THRESH_MEM_FILE` are compile-time parameters (empty string by default, since as top-level there's nothing above it to supply real values — that was previously the testbenches' job). User edited these directly in their copy of `snn_top.v` to point at real file paths and picked `threshold_ambient.mem` for this build. As before: **this means one bitstream classifies exactly one pre-selected audio sample** — there's still no runtime input path for live/varying audio, a design decision to make explicitly before Phase 7 if a live demo is wanted.

7. Patent-oriented notes (per your request — preliminary only, not a claim of novelty)

Design decision	Existing/ common approach	Proposed approach here	Advantage	Status
Presynaptic-spike-gated MAC	Dense MAC every cycle regardless of spike	Skip weight-row fetch/ accumulate entirely when input neuron didn't spike	Fewer BRAM reads & adds, scaling with measured sparsity (~5–14% active per your Section 10 data)	Architecture only — not yet implemented/ measured in RTL

Single shared deterministic LFSR across all 640×50 draws	Per-neuron RNG streams, or on-the-fly software RNG	One LFSR reused row-major across all (t,n) — cheap (1 LFSR total, not 640) but Python-reproducible	Verified bit-exact (§4.2); resource-minimal vs. per-neuron RNG	RTL done for the LFSR itself
Parameterized HIDDEN_N	Fixed-topology RTL matched to one trained model	Generic width, driven by the fact your own topology changed once already (64→128) mid-project	Avoids costly re-spin when model selection criteria change again	Design decision only

Prior-art questions still open (not investigated yet): event-driven/ spike-gated MAC skipping is a long-studied technique in SNN accelerator literature; any novelty claim would need to rest on a *specific* combination (e.g. this exact LFSR-based reproducibility scheme tying software and hardware RNG, if that turns out to be unusual) rather than on spike-gating alone. This needs a real prior-art search before any claim is made — flagging per your explicit instruction not to assert patentability without support.

8. Immediate next actions (proposed order) — updated Session 11

1. ~~Pick path A/B, get biases~~ — DONE.

2. ~~Implement rate_encoder.v~~ — DONE (§4.8–4.9).
3. ~~Implement weight memory modules~~ — DONE (§4.3–4.4, §4.6).
4. ~~Design + implement sigmoid_lut.v~~ — DONE (§4.10).
5. ~~Implement spike_gated_mac.v~~ — DONE (§4.11).
6. ~~Implement combine_scale.v + hidden_neuron.v~~ — DONE (§4.12).
7. ~~Implement output_layer.v~~ — DONE (§4.13).
8. ~~Implement snn_top.v integration~~ — DONE (§4.14).
9. ~~Fix missing input standardization~~ — DONE, verified at RTL level (§5.0).
10. ~~Run full 48 sample batch through actual RTL simulation~~ — DONE (§4.15).
11. ~~First Quartus synthesis attempt~~ — DONE (§6): 0 errors, fit succeeded.
12. ~~Add neuro.sdc, confirm real timing~~ — DONE: **50MHz PASSES with positive slack in every corner** (§6.2). Real, meaningful result.
13. **User:** apply the corrected `combine_scale.v` (second attempt — first one didn't actually work, see §6.1) and recompile once more, to get a fully clean warning log.
14. **User:** check Fitter → Resource Usage Summary for `spike_storage`'s specific LE contribution, to confirm or rule out the resource-usage hypothesis in §6.3.
15. Since timing already comfortably passes at 50MHz: decide whether `spike_gated_mac.v`'s sequential-MAC speed (§4.11) is even worth addressing now (it's a resource/inference-time concern, not currently a timing-closure blocker), or whether the design can proceed toward pin planning / board bring-up as-is.
16. **Design decision still open, not yet made:** live/varying audio input vs. one-sample-per-bitstream (§6.5) — needs deciding before Phase 7 if a live demo is wanted.
17. Phase 7: pin assignment, program the DE2-115, bring-up, demo.

This report will be updated in place as work continues. Historical results (including the superseded 64-hidden-neuron plan) are preserved above rather than deleted, per your Section 37/Appendix philosophy.